JCUIScaler 1.0 – Specyfikacja Techniczna

FINAL – wersja R1-09

1. Wprowadzenie

JCUIScaler jest biblioteką automatycznego skalowania interfejsów użytkownika dla Lazarus LCL, z nieoficjalną kompatybilnością na poziomie kodu źródłowego z Delphi VCL (sekcja 12.3). Biblioteka wykorzystuje deterministyczny model referencyjny: każde przeliczenie wykonywane jest wyłącznie względem stanu oryginalnego formularza i kontrolek, zarejestrowanego przy pierwszym wywołaniu ScalerOnShow po ScalerInit. Dzięki temu wykluczona jest akumulacja błędów zaokrągleń przy wielokrotnym skalowaniu.

Bibliotekę można zintegrować ręcznie — deweloper dodaje trzy wywołania do zdarzeń formularza — albo za pomocą komponentu TJCScaler (sekcja 12.2), który wykonuje te same wywołania automatycznie po umieszczeniu go na formularzu. Rdzeń działa samodzielnie; komponent jest wyłącznie warstwą integracyjną.

2. Architektura

Rdzeń biblioteki oparty jest na strukturze typu object (TJCUIScaler). Nie wykorzystuje modelu class jako podstawy działania. Architektura została zoptymalizowana pod kątem prostoty, przewidywalności oraz bezpieczeństwa działania — obiekt żyje na stosie i nie wymaga jawnego zwalniania.

2.1 Moduły systemu

Kod rdzenia jest podzielony na cztery pliki, przy czym trzy pliki .inc są włączane dyrektywą {$I} do JCUIScaler.pas i mają dostęp do wszystkich prywatnych pól obiektu. Komponent TJCScaler, jego rejestracja w IDE i pakiety Lazarusa znajdują się w osobnych plikach i nie zmieniają rdzenia. Nazwy plików należy zachować dokładnie w podanej postaci — w systemach Linux i macOS wielkość liter ma znaczenie:

Plik	Zawartość
JCUIScaler.pas	Główny moduł: deklaracja TJCUIScaler, publiczne API, ScaleForm, ScaleControl, ScalerInit, ScalerOnShow, ScalerOnResize.
JCUIScaler.capture.inc	Implementacja CaptureOriginals – iteracyjne DFS, budowa modelu referencyjnego.
JCUIScaler.controls.inc	Implementacja ApplyControlExceptions – wyjątki skalowania dla komponentów specjalnych (gridy, ListView, TreeView, StatusBar, Splitter).
JCUIScaler.find.inc	Pomocnicze funkcje Find*OrigIndex: FindDBGridOrigIndex, FindStrGridOrigIndex, FindCtrlOrigIndex, FindListViewOrigIndex, FindStatusBarOrigIndex, FindTreeViewOrigIndex – wyszukiwanie rekordów w modelu referencyjnym.
JCUIScalerComponent.pas	Komponent TJCScaler (LCL) — warstwa integracyjna wywołująca metody publiczne TJCUIScaler (sekcja 12.2).
design/JCUIScalerReg.pas	Rejestracja TJCScaler na palecie komponentów (strona JCUI) wraz z ikonami (jcuiscaler_icons.res).

packages/lazarus/*.lpk	jcuiscaler_rt (RunTime: rdzeń + komponent) oraz jcuiscaler_dsgn (DesignTime: rejestracja). Źródła pobierane z katalogu source bez kopiowania.

3. Model referencyjny

Model referencyjny jest zestawem dynamicznych tablic rekordów wypełnianych raz podczas wywołania CaptureOriginals (przy pierwszym ScalerOnShow po ScalerInit, gdy formularz jest już przeskalowany przez LCL). Przechowuje stan początkowy formularza i każdej kontrolki i stanowi jedyne źródło danych wykorzystywane podczas skalowania — nigdy bieżący stan ekranu.

Zdefiniowane typy rekordów:

Rekord	Przechowywane dane
TCtrlOrig	Każda kontrolka: Left, Top, Width, Height, Font.Size. Tablica CtrlOrigs – baza dla ogólnego skalowania.
TDBGridOrig	TDBGrid: szerokości kolumn (ColWidths[]), oryginalna szerokość klienta grida (OrigGridWidth).
TStringGridOrig	TStringGrid: ColWidths[], OrigGridWidth – analogicznie do TDBGridOrig.
TListViewOrig	TListView: szerokości kolumn (ColWidths[]), OrigWidth, OrigFontSize.
TTreeViewOrig	TTreeView: OrigFontSize. Pola OrigItemHeight i OrigIndent zarezerwowane (aktualnie 0) – patrz sekcja 6.
TStatusBarOrig	TStatusBar: szerokości paneli (PanelWidths[]), OrigHeight.
TGroupBoxOrig	TGroupBox: OrigFontSize, OrigCaptionOffset (aktualnie 0, best-effort).
TSplitterOrig	TSplitter: OrigWidth, OrigHeight.
TBorderOrig	Kontrolki TWinControl z ustawionym bitem Tag JCSCALER_BORDER_TAG ($10000000): oryginalne wartości BorderSpacing.Left/Top/Right/Bottom. Tylko LCL.

> **UWAGA:** Model referencyjny jest wypełniany jednorazowo przy pierwszym ScalerOnShow po ScalerInit. Dynamiczne tworzenie kontrolek po tej chwili nie jest automatycznie rejestrowane. Wymagana jest ręczna reinicjalizacja (ponowne wywołanie ScalerInit, a następnie ScalerOnShow).

4. CaptureOriginals – budowa modelu referencyjnego

Procedura CaptureOriginals realizuje iteracyjne przeszukiwanie DFS drzewa kontrolek formularza przy użyciu własnego stosu (TWinControlStack). Unika klasycznej rekurencji, co eliminuje ryzyko przepełnienia stosu przy głęboko zagnieżdżonych strukturach.

Algorytm:

```
1. Inicjalizacja stosu (INITIAL_STACK_SIZE = 256 elementów, rośnie dynamicznie).
```

```
2. Na stos trafia korzeń: FForm.
3. Główna pętla (while Top >= 0):
   a. Zdejmij TWinControl ze stosu.
   b. Wykryj i zarejestruj komponenty specjalne
      (TDBGrid, TStringGrid, TListView, TTreeView,
       TStatusBar, TGroupBox, TSplitter, TBorderOrig).
   c. Dla każdej kontrolki-dziecka (Root.Controls[i]):
      – zawsze rejestruj rekord TCtrlOrig (Left/Top/Width/Height/FontSize),
      – jeśli dziecko jest TWinControl: wrzuć na stos.
4. Zabezpieczenie: pętla kończy się po MAX_CONTROL_COUNT = 8192 kontrolkach.
```

Ważna konsekwencja architektury: TToolButton jest wykluczony ze skalowania na etapie ScaleControl (sprawdzenie is TToolButton przed wyszukaniem rekordu), nawet jeśli trafi do modelu referencyjnego. TSpeedButton nie jest wykluczony i skalowany jest jak każda inna kontrolka.

5. Algorytm skalowania

5.1 Wyliczanie współczynników

Każde wywołanie ScalerOnResize uruchamia ScaleForm, która wylicza dwa niezależne współczynniki:

```
overallScaleX := FForm.Width  / FormOriginalWidth;
overallScaleY := FForm.Height / FormOriginalHeight;
```

Skalowanie nie następuje, gdy formularz jest mniejszy niż:

```
MIN_FORM_WIDTH  = 200 px
MIN_FORM_HEIGHT = 200 px
```

5.2 ScaleControl – ogólne skalowanie

Dla każdej kontrolki pozycja i rozmiar wyliczane są z modelu referencyjnego (nigdy z wartości bieżących):

```
newL := LocalRound(co.OrigLeft   * overallScaleX);
newT := LocalRound(co.OrigTop    * overallScaleY);
newW := LocalRound(co.OrigWidth  * overallScaleX);
newH := LocalRound(co.OrigHeight * overallScaleY);
ctrl.SetBounds(newL, newT, newW, newH);
```

Dla TPanel przed SetBounds zapamiętywane są BevelWidth i BorderWidth, a po skalowaniu przywracane (gdy SkipBevelScaling = True) — zapobiega to optycznemu powiększaniu bevelów.

Po zastosowaniu głównego skalowania procedura rekurencyjnie skaluje dzieci (TWinControl.Controls[]), a następnie wywołuje ApplyControlExceptions dla wyjątków specjalnych.

5.3 ApplyControlExceptions – wyjątki komponentów specjalnych

Po ogólnym skalowaniu każda kontrolka przechodzi przez ApplyControlExceptions, która stosuje dedykowane algorytmy dla komponentów o złożonej strukturze wewnętrznej. Sekcja BorderSpacing wykonywana jest jako pierwsza i bez wyjścia z procedury, dzięki czemu obejmuje również komponenty z dedykowaną obsługą, a gridy, ListView i StatusBar liczą szerokości kolumn już po ustaleniu ostatecznego BorderSpacing:

Komponent	Opis algorytmu

TDBGrid	Skalowanie proporcjonalne szerokości kolumn względem aktualnej szerokości klienta grida. Ostatnia kolumna korygowana tak, by suma = totalWidth (z odjęciem GridLastWOffset).
TStringGrid	Identyczny algorytm jak TDBGrid, ale przez właściwość ColWidths[].
TListView	Skalowanie kumulatywne: pozycja prawej krawędzi każdej kolumny wyliczana jako LocalRound(cumOrig * scaleFactor). Ostatnia kolumna wypełnia pozostałą szerokość klienta.
TStatusBar	Skalowanie kumulatywne szerokości paneli (analogicznie do TListView).
TTreeView	Skalowanie Indent (overallScaleX). DefaultItemHeight skalowany wyłącznie pod {$IFDEF LCLGTK2}. LCL-owy TTreeView nie posiada właściwości ItemHeight (do R1-03 kod odwoływał się do niej i nie kompilował się pod GTK2).
TSplitter	Skaluje Width (alLeft/alRight) lub Height (alTop/alBottom) zgodnie z wyrównaniem.
BorderSpacing	Kontrolki TWinControl z bitem Tag JCSCALER_BORDER_TAG mają skalowane BorderSpacing.Left/Top/Right/Bottom względem wartości oryginalnych. Tylko LCL — w Delphi bit jest ignorowany.

6. Obsługiwane komponenty

Ogólne skalowanie (bounds) obejmuje wszystkie kontrolki zarejestrowane w CtrlOrigs. Specjalna obsługa dotyczy:

TPageControl, TStringGrid, TDBGrid, TListView, TTreeView, TStatusBar, TSplitter, TBevel (opcjonalne wykluczenie), TNoteBook.

Komponenty wykluczone ze skalowania: TToolButton – pomijany w ScaleControl niezależnie od zawartości modelu referencyjnego. TSpeedButton nie jest wykluczony (korekta specyfikacji w R1-04; wykluczenie dotyczyło wyłącznie problemów obserwowanych z TToolButton).

TGroupBox: rejestrowany w modelu referencyjnym (OrigFontSize, OrigCaptionOffset), ale nie posiada aktywnej sekcji w ApplyControlExceptions w wersji 1.0. Obsługa zostanie dodana wraz z implementacją skalowania czcionek.

6.1 Ograniczenia TTreeView

Pola OrigItemHeight i OrigIndent w TTreeViewOrig są aktualnie przechowywane jako 0 (jawna decyzja projektowa RC1, oznaczona w kodzie komentarzem FS3). Skalowanie Indent jest aktywne. DefaultItemHeight skalowany jest wyłącznie pod GTK2; zachowanie tej właściwości w innych widgetsetach LCL nie zostało zweryfikowane. Przy wartościach 0 obie operacje nie mają w RC1.2 efektu. Scenariusz jest zaplanowany do rozwiązania w przyszłej wersji.

7. Skalowanie czcionek

Skalowanie czcionek nie jest zaimplementowane w wersji 1.0. Próba implementacji ujawniła głębokie zależności między rozdzielczościami, DPI i platformami (w tym różnicami między wersjami Windows), które

uniemożliwiają deterministyczne i przewidywalne działanie na wszystkich obsługiwanych konfiguracjach. Podjęta została decyzja o rollbacku i odłożeniu tego obszaru poza wersję 1.0.

Pole OrigFontSize jest rejestrowane w modelu referencyjnym (TCtrlOrig, TListViewOrig, TTreeViewOrig, TGroupBoxOrig) i przywracane przez RestoreInitialScale — infrastruktura jest gotowa na przyszłą implementację gdy problem zależności platformowych zostanie rozwiązany.

> **UWAGA:** Skalowanie czcionek jest świadomie wyłączone w wersji 1.0. Status: do rozważenia w przyszłej wersji po rozwiązaniu problemów cross-platform.

8. Obsługa zdarzeń i metody publiczne

8.1 ScalerInit

```
procedure ScalerInit(AForm: TForm)
```

Inicjalizuje bibliotekę. Zapamiętuje referencję do formularza i ustawia wartości domyślne właściwości (SkipBevelScaling = True, ResetOnShow = False). Model referencyjny, w tym oryginalne wymiary i położenie formularza (FormOriginalWidth, FormOriginalHeight, FormOriginalLeft, FormOriginalTop), budowany jest dopiero przy pierwszym wywołaniu ScalerOnShow.

Wywołanie: FormCreate.

> **UWAGA:** Właściwości SkipBevelScaling i ResetOnShow należy ustawiać po ScalerInit — ScalerInit nadpisuje je wartościami domyślnymi. Domyślne wartości: SkipBevelScaling = True, ResetOnShow = False.

8.2 ScalerOnShow

```
procedure ScalerOnShow
```

Przy pierwszym wywołaniu po ScalerInit buduje model referencyjny (CaptureOriginals). Przy kolejnych wywołaniach wywołuje RestoreInitialScale, jeśli ResetOnShow = True; w pozostałych przypadkach nie wykonuje żadnej akcji.

Wywołanie: FormShow.

> **UWAGA:** Wywołanie jest obowiązkowe. Bez ScalerOnShow model referencyjny nie zostanie zbudowany, a ScalerOnResize nie wykona skalowania.

8.3 ScalerOnResize

```
procedure ScalerOnResize
```

Główna metoda skalowania – wywołuje ScaleForm, która przelicza geometrię całego drzewa kontrolek.

Wywołanie: FormResize.

8.4 RestoreInitialScale (publiczna)

Publiczny mechanizm awaryjny: przywraca formularz do stanu z chwili budowy modelu referencyjnego. Metoda jest wywoływana automatycznie przez ScalerOnShow, gdy ResetOnShow = True, i jest dostępna dla kodu aplikacji jako radykalny reset w sytuacjach nietypowych, których nie da się z góry zdefiniować (np. specyficzne zachowanie widgetsetu lub kontrolki zewnętrznej).

Działanie: przed zbudowaniem modelu (przed pierwszym ScalerOnShow) metoda nie wykonuje żadnej akcji; nie działa również, gdy formularz ma już oryginalny rozmiar i położenie. W pozostałych przypadkach ustawia formularz na oryginalne wymiary i położenie (FormOriginalLeft/Top/Width/Height), a pozycje i rozmiary kontrolek zarejestrowanych w modelu referencyjnym (CtrlOrigs) cofa proporcjonalnie odwrotnymi

współczynnikami ScaleBackX/Y (brute force — decyzja F-01). Kontrolki spoza modelu nie są modyfikowane bezpośrednio. Przywracane są także wartości Font.Size z OrigFontSize (bez efektu w 1.0, przygotowanie pod przyszłe skalowanie czcionek).

Skutki widoczne dla użytkownika: okno zmienia położenie i rozmiar. Jeśli metoda zostanie wywołana tuż po wyjściu ze stanu zmaksymalizowanego, gdy formularz nadal raportuje rozmiar zmaksymalizowany, formularz przechodzi przez przełączenie WindowState (wsMaximized → wsNormal).

Spójność ze stanem modelu: zmiana rozmiaru formularza wywołuje OnResize, a więc ScalerOnResize, który przelicza wszystkie kontrolki z modelu przy skali 1. Stan po resecie jest identyczny ze stanem pierwszego pokazania — potwierdzone na GTK2 (porównanie zrzutów co do piksela) i na Windows (Analiza Statyczna, F-22 zamknięte).

> **UWAGA:** RestoreInitialScale nie obejmuje scenariuszy Minimize/Restore oraz Maximize/Restore. Obsługa tych przypadków zależy od zachowania widgetsetu LCL i nie gwarantuje wywołania OnShow.

9. Właściwości i stałe publiczne

9.1 Właściwości

Właściwość	Typ	Domyślna	Opis
SkipBevelScaling	Boolean	True	Gdy True: BevelWidth i BorderWidth paneli TPanel są przywracane do oryginalnych wartości po każdym skalowaniu. Zapobiega optycznemu powiększaniu ramek.
ResetOnShow	Boolean	False	Gdy True: wywołanie ScalerOnShow przywraca formularz do oryginalnego rozmiaru i położenia (przez RestoreInitialScale).

9.2 Publiczne stałe

Stała	Wartość	Opis
MIN_FORM_WIDTH	200	Minimalna szerokość formularza przy której skalowanie jest aktywne.
MIN_FORM_HEIGHT	200	Minimalna wysokość formularza przy której skalowanie jest aktywne.
JCSCALER_BORDER_TAG	$10000000	Bit który należy ustawić w Tag kontrolki aby włączyć automatyczne skalowanie jej BorderSpacing. Dotyczy kontrolek TWinControl; tylko LCL (w Delphi ignorowany). Przykład: MyPanel.Tag := MyPanel.Tag or JCSCALER_BORDER_TAG;

9.3 LocalRound (prywatna metoda)

Wszystkie obliczenia pikselowe w bibliotece używają prywatnej metody LocalRound zamiast wbudowanej funkcji Round. LocalRound implementuje Floor(V + 0.5) — deterministyczne zaokrąglenie wartości *.5 zawsze w górę (w stronę +∞), niezależnie od trybu zaokrąglania. Do R1-03 stosowane było Trunc(V + 0.5), które dla wartości dodatnich daje identyczny wynik, ale błędnie zaokrągla wartości ujemne (np. -1,7 → -1 zamiast -2; dotyczy m.in. kontrolek w przewiniętym TScrollBox). Wbudowana funkcja Round stosuje bankierskie zaokrąglenie (round-half-to-even), które może dawać asymetryczne wyniki pikselowe przy wielokrotnym skalowaniu.

10. Założenia projektowe i ograniczenia

Założenie	Szczegóły
Skalowanie względem modelu referencyjnego	Eliminuje akumulację błędów – każde przeliczenie wychodzi od stanu oryginalnego.
Brak zależności od DPI systemowego	Biblioteka nie odczytuje DPI – skaluje wyłącznie na podstawie geometrii okna.
Rdzeń oparty na object	TJCUIScaler żyje na stosie. Brak konieczności Create/Free.
Brak modyfikacji widgetsetów i LCL	Biblioteka nie zmienia żadnych mechanizmów LCL.
Dynamiczne tablice tylko dla modelu	Tablice rekordów są alokowane jednorazowo w CaptureOriginals (pierwszy ScalerOnShow) i nie rosną w czasie skalowania.
Brak obsługi dynamicznych kontrolek	Kontrolki tworzone po zbudowaniu modelu nie są automatycznie rejestrowane. Wymagane ponowne ScalerInit i ScalerOnShow.
Brak skalowania czcionek (1.0)	Rollback z powodu głębokich zależności DPI/platforma/wersja systemu. Do rozważenia w przyszłej wersji.
TToolButton wykluczony	Skalowanie TToolButton jest jawnie pomijane w ScaleControl. TSpeedButton jest skalowany normalnie.
TListView na ukrytej zakładce	Natywny TListView na ukrytej zakładce podaje ClientWidth bez pionowego paska przewijania. Kolumny skalowane w tym czasie są szersze o szerokość paska do najbliższej zmiany rozmiaru formularza. Zalecenie: ScrollBars = ssVertical (sekcja 13.1). Gridy LCL nie są dotknięte.
Anchors = [] przy innym DPI	Kontrolki bez kotwic są przesuwane przez LCL podczas automatycznego skalowania DPI (DPI różne od DesignTimePPI), zanim powstanie model referencyjny. Zalecenie: domyślne Anchors = [akLeft, akTop] (sekcja 13.1).
TJCScaler — podmiana OnShow w czasie działania	Komponent łańcuchuje OnShow formularza w Loaded (LCL nie ma listy handlerów OnShow). Jeśli aplikacja przypisze nowy OnShow w czasie działania (także w FormCreate), należy następnie wywołać Attach.

11. Publiczne API – podsumowanie

Sygnatura	Opis
procedure ScalerInit(AForm: TForm)	Inicjalizacja – zawsze pierwsza. Ustawia wartości domyślne; model referencyjny budowany jest przy pierwszym ScalerOnShow.
procedure ScalerOnShow	Przy pierwszym wywołaniu buduje model referencyjny, później opcjonalnie przywraca stan (ResetOnShow). Wywoływać z FormShow — obowiązkowo.
procedure ScalerOnResize	Skaluje cały formularz. Wywoływać z FormResize.
procedure RestoreInitialScale	Publiczny mechanizm awaryjny — przywraca stan z chwili budowy modelu referencyjnego. Wywoływana przez ScalerOnShow przy ResetOnShow = True lub bezpośrednio z kodu aplikacji. Patrz 8.4.
property SkipBevelScaling: Boolean	Ochrona bevelów paneli TPanel. Domyślnie True.
property ResetOnShow: Boolean	Przywrócenie oryginalnego stanu przy OnShow. Domyślnie False.
JCSCALER_BORDER_TAG = $10000000	Stała do ustawienia w Tag kontrolki — włącza skalowanie BorderSpacing.

12. Integracja z formularzem

12.1 Integracja ręczna

Skopiować pliki biblioteki (JCUIScaler.pas, JCUIScaler.capture.inc, JCUIScaler.controls.inc, JCUIScaler.find.inc) do katalogu projektu, dodać jednostkę JCUIScaler do sekcji uses i dodać pole FScaler: TJCUIScaler do klasy formularza.

```
type
  TMainForm = class(TForm)
  private
    FScaler: TJCUIScaler;
  end;

procedure TMainForm.FormCreate(Sender: TObject);
begin
  FScaler.ScalerInit(Self);
  // Opcjonalnie, po ScalerInit (ScalerInit ustawia wartości domyślne):
  // FScaler.SkipBevelScaling := True;   // domyślnie True
  // FScaler.ResetOnShow := False;        // domyślnie False
end;

procedure TMainForm.FormShow(Sender: TObject);
begin
  FScaler.ScalerOnShow;
end;

procedure TMainForm.FormResize(Sender: TObject);
begin
  FScaler.ScalerOnResize;
end;
```

UWAGA: Kolejność jest istotna: ScalerInit w FormCreate, właściwości ustawiane po ScalerInit, ScalerOnShow w FormShow, ScalerOnResize w FormResize. Do pierwszego ScalerOnShow ScalerOnResize nie wykonuje skalowania.

12.2 Integracja komponentowa — TJCScaler

TJCScaler (unit JCUIScalerComponent) jest warstwą integracyjną dla LCL: komponent umieszczony na formularzu wykonuje automatycznie trzy wywołania rdzenia. Nie zastępuje rdzenia i nie zmienia jego działania — zawiera obiekt TJCUIScaler i wywołuje wyłącznie jego metody publiczne. Instalacja: pakiet packages/lazarus/jcuiscaler_dsgn.lpk (wymaga jcuiscaler_rt.lpk); komponent pojawia się na stronie palety JCUI.

Nazwa klasy: TJCScaler, aby domyślna nazwa instancji na formularzu była krótka (JCScaler1). Obiekt rdzenia zachowuje nazwę TJCUIScaler — jest używany produkcyjnie z integracją ręczną i jego nazwa nie może się zmienić bez zerwania kompatybilności.

Działanie: Loaded wywołuje Attach — ScalerInit dla formularza właściciela, a następnie przypisanie właściwości. OnShow formularza jest łańcuchowany; na każde pokazanie wywoływany jest najpierw OnShow aplikacji, potem ScalerOnShow — w tej samej kolejności co przy integracji ręcznej. Zmiana rozmiaru jest obsługiwana przez AddHandlerOnResize (lista handlerów LCL); OnResize aplikacji pozostaje nietknięty i jest wywoływany jako pierwszy. Wyniki komponentu są identyczne co do piksela z integracją ręczną.

Właściwości (published, powiązane z obiektem rdzenia): SkipBevelScaling (domyślnie True) i ResetOnShow (domyślnie False). Można je ustawić w Object Inspectorze lub zmieniać w kodzie w dowolnym momencie — po Attach są odczytywane i zapisywane bezpośrednio w obiekcie TJCUIScaler.

Zdarzenia (published, domyślnie puste; Sender = komponent): BeforeShow, AfterShow, BeforeResize, AfterResize. Kolejność przy pokazaniu formularza: OnShow formularza → BeforeShow → ScalerOnShow → AfterShow. Kolejność przy zmianie rozmiaru: OnResize formularza → BeforeResize → ScalerOnResize → AfterResize. Przy pierwszym pokazaniu BeforeShow działa przed zbudowaniem modelu referencyjnego. AfterResize jest miejscem na dostosowania, których skaler nie wykonuje (czcionki, kontrolki spoza modelu, kontrolki rysowane samodzielnie); geometria kontrolek z modelu ustawiona w BeforeResize zostanie nadpisana przez skaler.

Rozszerzanie w klasie potomnej: chronione metody wirtualne DoShow i DoResize (domyślnie: zdarzenie Before*, skaler, zdarzenie After*) oraz chroniona właściwość Attached. Metody publiczne: Attach (dla komponentu tworzonego w kodzie lub po podmianie OnShow w czasie działania), Rescale (= ScalerOnResize, np. w TPageControl.OnChange na Windows — sekcja 13.1) oraz RestoreInitialScale.

Ograniczenia: jeden komponent na formularz (aktywny jest pierwszy); właścicielem musi być TForm — w przeciwnym razie (ramka, moduł danych) komponent pozostaje nieaktywny; tylko LCL (wykorzystuje listy handlerów LCL). Projekt demonstracyjny celowo używa integracji ręcznej i nie wymaga instalacji pakietu.

12.3 Delphi (nieoficjalnie)

Kod źródłowy JCUIScaler kompiluje się również w Delphi VCL — potwierdzone w Delphi 13.2 (Win64): kompilacja i podstawowe działanie, także z kontrolkami zewnętrznymi (TMS UI Pack PageControl, RxDBGrid). Wsparcie ma charakter nieoficjalny i jest ograniczone do integracji ręcznej identycznej jak w sekcji 12.1: te same cztery pliki, ta sama sekcja uses, te same trzy wywołania. Pakiet dla Delphi nie jest dostarczany — pakiet .lpk dotyczy wyłącznie Lazarusa.

Architektura oparta na object pozostaje bez zmian. Różnice między FPC/LCL a Delphi/VCL są obsłużone kodem neutralnym (klasa dostępowa TJCControlAccess dla protected TControl.Font w VCL, jawna inicjalizacja zmiennych lokalnych) oraz trzema blokami {$IFDEF FPC}: dyrektywą trybu kompilatora oraz capture i apply BorderSpacing. Moduły VCL rozwiązywane są przez standardowe unit scope names projektu VCL.

Ograniczenia w Delphi: JCSCALER_BORDER_TAG jest ignorowany (VCL nie ma BorderSpacing; Margins ma inną semantykę i nie jest emulowany); zmiana DPI monitora (R2-01) nie jest obsługiwana — skalowanie per-monitor VCL może nakładać się na działanie skalera w konfiguracjach wielomonitorowych; TJCUIScaler należy deklarować jako pole formularza (jak w sekcji 12.1); TSplitter w VCL dziedziczy po TGraphicControl, więc otrzymuje wyłącznie ogólne skalowanie położenia i rozmiaru (bez dedykowanej obsługi z sekcji 5.3).

Projekt demonstracyjny działa wyłącznie w Lazarusie: poza brakiem plików .dfm opiera się na elementach specyficznych dla LCL (pliki .lfm/.lpi, pakiet DateTimeCtrls, nazwy właściwości LCL w formularzach).

Komponent TJCScaler i pakiety .lpk są dostępne wyłącznie dla Lazarusa; w Delphi stosuje się integrację ręczną.

Pliki źródłowe mają końce linii CRLF (wymagane przez RAD Studio, akceptowane przez Lazarusa); plik .gitattributes utrzymuje je w repozytorium.

13. Kompatybilność

Element	Wymaganie
Lazarus	3.0+ (testowane: 3.0 — Linux/GTK2, 4.8 — Windows, 4.8 — macOS)
Free Pascal	3.2.2+
Systemy operacyjne	Windows, Linux, macOS
Widgetsety	Win32/Win64, GTK2 (DefaultItemHeight TreeView) – z zastrzeżeniami opisanymi w sekcji 6.1; Cocoa (macOS 27) – przetestowane; Qt – nietestowane
Delphi (nieoficjalnie)	Delphi 13.2 (Win64), VCL — zweryfikowane: kompilacja i podstawowe działanie; tylko integracja ręczna, bez pakietu. Patrz sekcja 12.3.

13.1 Zalecenia konfiguracji kontrolek

Kotwice. Kontrolki skalowane przez JCUIScaler powinny mieć domyślne Anchors = [akLeft, akTop]. Wyczyszczenie kotwic (Anchors = []) powoduje, że przy DPI różnym od DesignTimePPI formularza LCL przesuwa kontrolki podczas automatycznego skalowania DPI — przed pierwszym ScalerOnShow. Skaler zapisuje wtedy w modelu referencyjnym już rozjechany układ i wiernie go zachowuje. Zweryfikowane na GTK2 przy 96 i 120 DPI (DesignTimePPI = 120).

Paski przewijania TListView (vsReport): zalecane ScrollBars = ssVertical. JCUIScaler dopasowuje kolumny do szerokości kontrolki, więc poziomy pasek przewijania nie jest potrzebny. Natywny TListView na ukrytej zakładce podaje ClientWidth bez pionowego paska; kolumny skalowane w tym czasie są szersze o szerokość paska do najbliższej zmiany rozmiaru formularza (Windows i GTK2; GTK2: 451 px na ukrytej zakładce, 435 px po jej pokazaniu). Przy ssVertical nadmiar pozostaje pod pionowym paskiem — przycinany jest tylko pusty koniec ostatniej kolumny i nie pojawia się poziomy pasek. Rozwiązanie nie wymaga kodu ani dodatkowych wywołań.

Gridy LCL (TStringGrid, TDBGrid) nie mają tego problemu: są rysowane przez LCL i same zarządzają paskami, więc ClientWidth jest poprawny również na ukrytej zakładce (GTK2: ssAutoBoth, ssVertical, ssBoth; 6 i 60 wierszy). Ustawienie ssVertical jest dla nich rozsądną wartością domyślną, ale nie jest wymagane.

Opcjonalnie, tylko Windows: dokładne szerokości kolumn natychmiast po zmianie zakładki uzyskuje się wywołaniem FScaler.ScalerOnResize w TPageControl.OnChange (przy komponencie: JCScaler1.Rescale). ScalerOnResize przelicza wszystko z modelu referencyjnego, więc wywołanie jest deterministyczne i

bezpieczne przy niezmienionym rozmiarze formularza. Na GTK2 nie działa: TTabSheet.OnShow i TPageControl.OnChange są wywoływane przed ułożeniem nowej zakładki.

GTK2: domyślne ScrollBars = ssBoth jest w widgetsecie GTK2 odwzorowywane jako GTK_POLICY_ALWAYS — oba paski są widoczne zawsze, niezależnie od zawartości.

Rozwiązania odrzucone (sprzeczne z założeniami biblioteki lub niewiarygodne): handler OnResize rejestrowany przez skaler na kontrolkach (skaler reagowałby na zdarzenia poza trzema jawnymi wywołaniami, a jego czas życia zostałby powiązany z kontrolkami); wywołanie odroczone (Application.QueueAsyncCall) — widoczne mrugnięcie; rezerwa na szerokość paska — na GTK2 GetSystemMetrics(SM_CXVSCROLL) = 16 px przy rzeczywistej szerokości paska 19 px.

13.2 Licencja

JCUIScaler jest rozpowszechniany na tej samej licencji co biblioteka LCL Lazarusa: GNU Library General Public License, wersja 2 (plik COPYING.LGPL.txt), z wyjątkiem dla linkowania statycznego (plik COPYING.modifiedLGPL.txt — tekst wyjątku identyczny z Lazarusem). Pliki LICENSE, COPYRIGHT i AUTHORS znajdują się w katalogu głównym.

Każdy plik źródłowy zawiera nagłówek z informacją o prawach autorskich (Copyright (c) 2025-2026 Jakub Cebula, Kraków) i odesłaniem do pliku licencji. Informacji o prawach autorskich ani nazwiska pierwotnego autora nie wolno usuwać z plików źródłowych ani z ich zmodyfikowanych wersji; nie zmienia to warunków licencji.

14. Zakres testów technicznych

Testy weryfikują następujące scenariusze:

- Resize (zmiana rozmiaru okna w dowolnym kierunku)
- Maximize/Restore
- PageControl (zakładki, przełączanie stron)
- Gridy (TStringGrid, TDBGrid – skalowanie kolumn, ostatnia kolumna)
- Struktury zagnieżdżone (kontrolki wewnątrz paneli, GroupBox)
- Regresja (brak dryfu po wielokrotnym resize)
- Kompilacja: FPC/LCL ze wszystkimi wspieranymi widgetsetami (w tym LCLGTK2) oraz Delphi VCL (Delphi 13.2, Win64 — kompilacja, podstawowe działanie, kontrolki zewnętrzne)
- Testy uruchomieniowe: DPI różne od DesignTimePPI (96 i 120 DPI), zmiana rozmiaru przy ukrytej zakładce (TListView), drugi formularz z ResetOnShow i JCSCALER_BORDER_TAG
- Komponent TJCScaler: zgodność co do piksela z integracją ręczną (komponent z pliku .lfm oraz tworzony w kodzie), kolejność zdarzeń, właściwości powiązane z obiektem, metody wirtualne
- Platformy: Windows 11 (Lazarus 4.8), macOS 27 (Lazarus 4.8, Cocoa), Linux (Lazarus 3.0, GTK2); Delphi 13.2 (Win64, nieoficjalnie)

15. Kierunki rozwoju

Obszar	Opis
Skalowanie czcionek	Do rozważenia po 1.0. Wymaga rozwiązania zależności DPI/platforma/wersja systemu. Infrastruktura (OrigFontSize) gotowa w modelu referencyjnym.
TGroupBox w ApplyControlExceptions	Powiązane ze skalowaniem czcionek — zostanie dodane razem z nim.

TreeView DefaultItemHeight	Rozwiązanie cross-widgetset dla skalowania DefaultItemHeight poza GTK2.
Reinicjalizacja po zmianie monitora	Obsługa scenariuszy przeniesienia okna między monitorami o różnej skali.
Delphi — zmiana DPI monitora	Obsługa przez OnBeforeMonitorDpiChanged / OnAfterMonitorDpiChanged (VCL) według hipotezy R2-01.
Pakiet OPM	Dystrybucja przez Online Package Manager.
Dokumentacja PL/EN	Polska wersja jako dokument źródłowy; tłumaczenie angielskie.

16. Wnioski z analizy statycznej RC1

Kod RC1 po sesji cleanup 2026-06-01 jest spójny architektonicznie i implementacyjnie. Wszystkie ustalenia analizy statycznej Rev D zostały rozstrzygnięte: część naprawiona w kodzie, część zamknięta świadomymi decyzjami projektowymi z udokumentowanym uzasadnieniem.

Model referencyjny jest kompletny i poprawnie wypełniany przez CaptureOriginals dla wszystkich obsługiwanych typów komponentów. Algorytmy skalowania kolumn (DBGrid, StringGrid, ListView, StatusBar) są deterministyczne i odporne na akumulację błędów zaokrągleń dzięki metodzie kumulatywnej z korekcją ostatniej kolumny i jednolitej funkcji LocalRound.

RC1.2 (2026-11-06, Static Analysis Rev G) usunęła trzy błędy niewykryte w rewizjach D–F: zapis pozycji kontrolki zamiast BorderSpacing w modelu referencyjnym (wraz z nieosiągalnością sekcji BorderSpacing dla komponentów z dedykowaną obsługą), brak kompilacji pod LCLGTK2 (nieistniejąca właściwość ItemHeight) oraz błędne zaokrąglanie wartości ujemnych w LocalRound. Wprowadzono także nieoficjalną kompatybilność źródłową z Delphi VCL. Specyfikacja została skorygowana w zakresie TSpeedButton (nie jest wykluczony) oraz momentu budowy modelu referencyjnego (pierwszy ScalerOnShow).

Testy uruchomieniowe z 2026-11-07 (GTK2, 96 i 120 DPI; Static Analysis Rev H) potwierdziły brak dryfu po cyklach zmiany rozmiaru, poprawne skalowanie BorderSpacing oraz wszystkich zakładek demo. Ujawniły dwa zachowania wymagające zaleceń konfiguracyjnych (sekcja 13.1): Anchors = [] przy DPI różnym od DesignTimePPI oraz TListView na ukrytej zakładce. Kod biblioteki nie wymagał zmian.

RC2 (2026-11-07, Static Analysis Rev I) wprowadza komponent TJCScaler, pakiety Lazarusa i pliki licencyjne. Rdzeń pozostał bez zmian w kodzie (zmiany wyłącznie w komentarzach: nagłówki licencyjne, opis RestoreInitialScale). Komponent sprawdzono względem ryzyk krytycznych architektury (Analiza Ryzyka Rev C) — jest zgodny.

Wersja 1.0: kod, pakiety, licencja oraz dokumentacja polska i angielska (z README) są kompletne. Biblioteka jest gotowa do publikacji w repozytorium publicznym i zgłoszenia do OPM. Kompilacja i podstawowe działanie w Delphi 13.2 zostały potwierdzone.

Demo zweryfikowano także na macOS 27 z Lazarusem 4.8 (Cocoa). Przetestowane platformy: Windows, macOS i Linux/GTK2 oraz — nieoficjalnie — Delphi 13.2.

17. Uwagi dotyczące dokumentacji

Polska wersja jest dokumentem źródłowym; wersja angielska jest jej tłumaczeniem. Numery wersji dokumentacji są niezależne od numerów wersji kodu.

Specyfikacja techniczna jest dokumentem aktualnym i aktualizowanym. Dokumenty uzupełniające — Analiza Statyczna, Analiza Ryzyk i RoadMap — znajdują się w podkatalogu Additional; opisują proces

rozwoju do wersji 1.0 i nie są na bieżąco aktualizowane. Odwołania do ustaleń F-xx w niniejszej specyfikacji dotyczą rejestru w Analizie Statycznej.

Zasady wprowadzania zmian, w tym obowiązujące zasady architektury, są utrzymywane w pliku CONTRIBUTING.md. Dalsze planowanie i nowe ustalenia są prowadzone w repozytorium projektu (zgłoszenia na GitHubie).